Programming Fundamentals Using C++

Huma Israr

Department Of Information Engineering Technology

National Skills University, Islamabad.

huma.israr@nsu.edu.pk

Fall 2024, BS-CS

Week 07

Lecture – 01

Lecture –02

Control Structures (Switch & goto)

Semester Calendar

Class Schedule

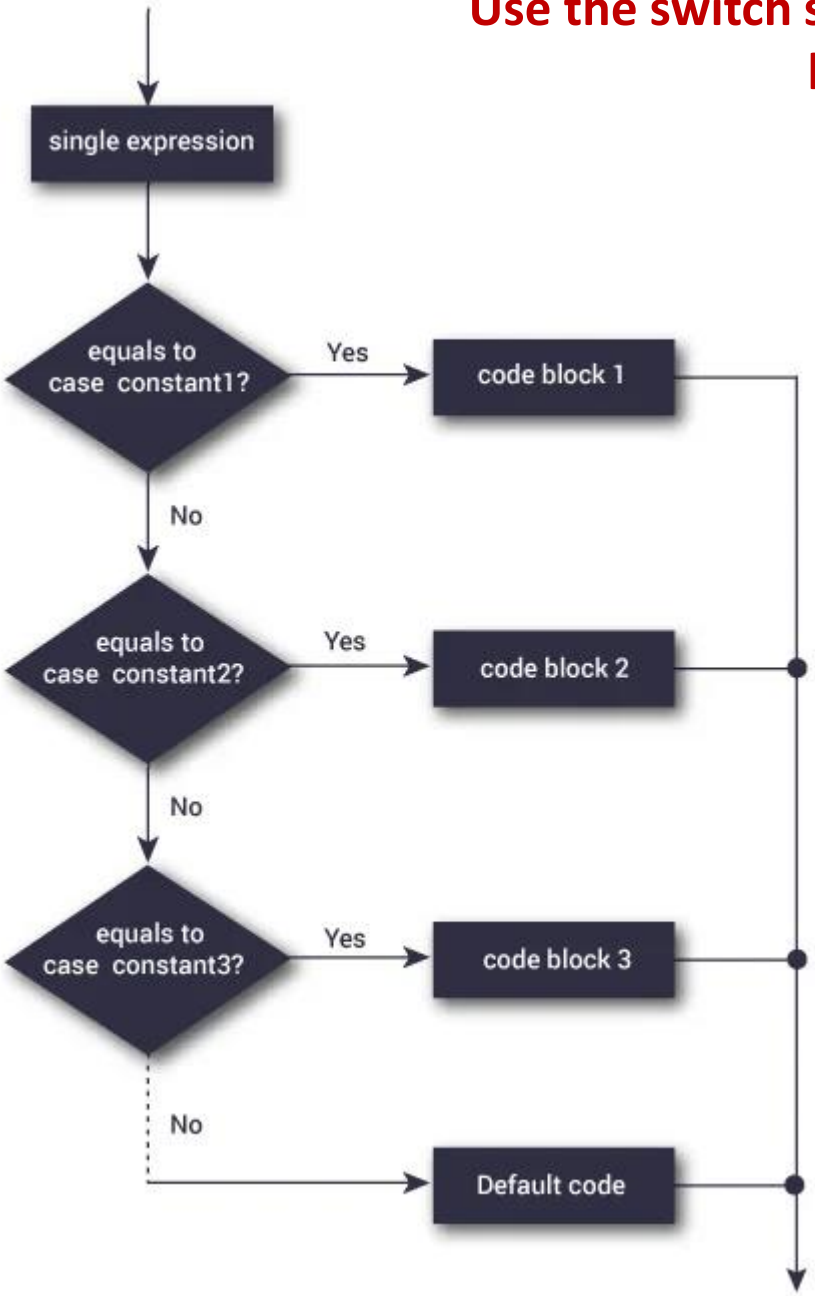
Lab Hours (Practice)

Tuesday (09:00 to 10:30)
Wednesday (09:30 to 12:30)
Wednesday (03:00 to 04:30)

C++ Switch Statements

- Switch case statement is used when we have multiple conditions and we need to perform different action based on the condition.
- When we have multiple conditions and we need to execute a block of statements when a particular condition is satisfied.
- In such case either we can use lengthy if..else-if statement or switch case.
- The problem with lengthy if..else-if is that it becomes complex when we have several conditions.
- The switch case is a clean and efficient method of handling such scenarios.

Use the switch statement to select one of many code blocks to be executed.



How it Works

- The switch expression is evaluated once
- The value of the expression is compared with the values of each case
- If there is a match, the associated block of code is executed
- The break and default keywords are optional, and will be described later in this chapter

Examples

```
#include <iostream>

using namespace std;

int main(){

int day = 2;

switch (day ) {

case  1:

   cout << "Saturday";

   break;

case  2:

   cout << "Sunday“;

   // break ;

}
```

```
#include <iostream>
using namespace std;
int main(){
  int num=5;
  switch(num+2) {
    case 1:
      cout<<"Case1: Value is: "<<num<<endl;
      break;
    case 2:
      cout<<"Case2: Value is: "<<num<<endl;
      break;
    case 3:
      cout<<"Case3: Value is: "<<num<<endl;
      break;
    default:
      cout<<"Default: Value is: "<<num<<endl;

} }
```

Default Statement

```
int day = 4;
switch (day) {
  case 6:
    cout << "Today is Saturday";
    break;
  case 7:
    cout << "Today is Sunday";
    break;
  default:
    cout << "Looking forward to the Weekend";
}
// Outputs "Looking forward to the Weekend"
```

Note:

- A switch statement can have an optional default case, which must appear at the end of the switch.
- The default case can be used for performing a task when none of the cases is true.
- No break is needed in the default case.

Break in Switch:

- Break statements are used when you want your program-flow to come out of the switch body. Whenever a break statement is encountered in the switch body, the execution flow would directly come out of the switch, ignoring rest of the cases. This is why you must end each case block with the break statement.

Switch Statement without Break

```
#include <iostream>
using namespace std;
int main(){
  int i=2;
  switch(i) {
    case 1: cout<<"Case1 "<<endl;
    case 2: cout<<"Case2 "<<endl;
    case 3: cout<<"Case3 "<<endl;
    case 4: cout<<"Case4 "<<endl;
    default: cout<<"Default "<<endl;
  }
  return 0;
}
```

Note:

- Since no break statement appears before the following case.
- Execution control is transferred to the second statement (Cout<<“Case2:) and continues in order through the rest of the body.

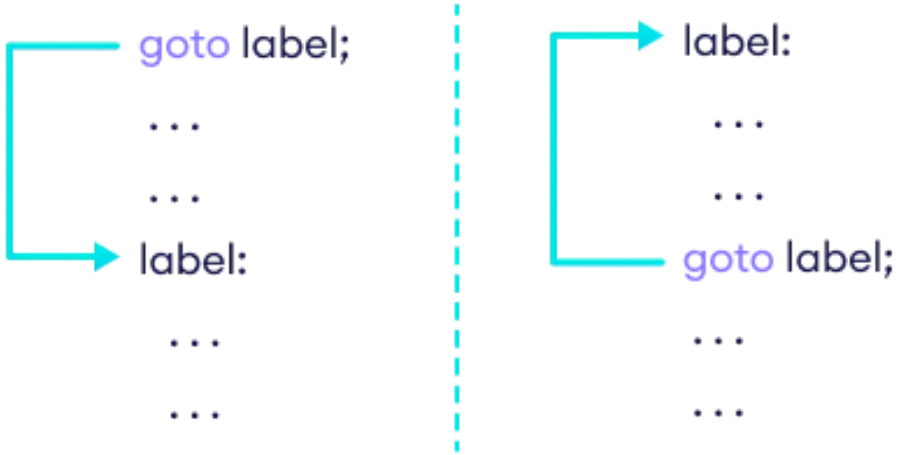
Can also use characters in switch case

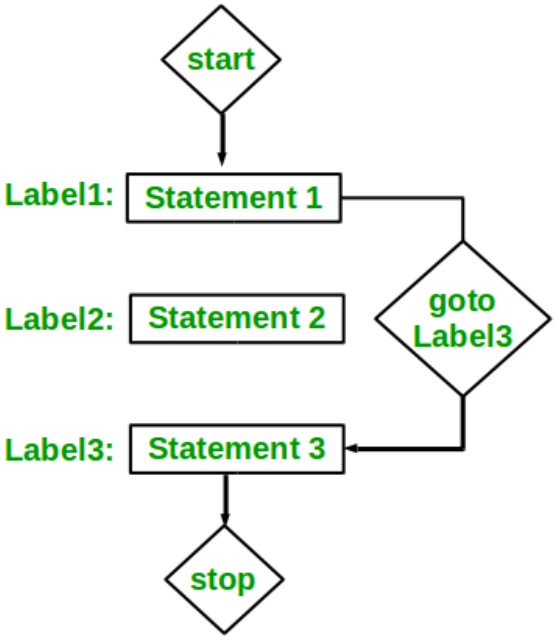
```
int main(){
  char ch='b';
  switch(ch) {
    case 'd': cout<<"Case1 ";
    break;
    case 'b': cout<<"Case2 ";
    break;
    case 'x': cout<<"Case3 ";
    break;
    case 'y': cout<<"Case4 ";
    break;
    default: cout<<"Default ";
  }
  return 0;  }
```

```
int main(){
  int i=2;
  switch(i) {
    case 1:  cout<<"Case1 "<<endl;
      break;
    case 2:   cout<<"Case2 "<<endl;
      break;
    case 3:    cout<<"Case3 "<<endl;
      break;
     default:
      cout<<"Default "<<endl;
   }
   return 0;
}
```

C++ goto statement

- In C++ programming, the goto statement is used for altering the normal sequence of program execution by transferring control to some other part of the program.
- Sytnax





Goto Example:

```
int num = 10;
if (num == 10)
{
  goto p;
}
else
{
  cout << "Hello I am not ten !! "<<endl;
}
p : cout << "Hello I am 10 and I am even." << endl;
```

C++ goto statement

- A goto statement provides an unconditional jump from the goto to a labeled statement in the same function.
- It allows the program's execution flow to jump to a specified location within the function.
- There are two ways to call the goto statement.

```
goto label;

//block of statements

label:  ;
```

```
label:  ;

//block of statements

goto  label;
```

```
//based on syntax 1
#include<iostream>
using namespace std;
int main()
{
int i, j;
i=2;j=5;
if(i>j)
goto iGreater;
else
goto jGreater;
iGreater:
cout<<i<<" is greater";
return 0;
jGreater:
cout<<j<<" is greater";
}
```

```
//based on Syntax 2

#include <iostream>
using namespace std;
// function to print numbers from 1 to 5
int main ()
{
int n = 1;
print:
cout << n << " ";
n++;
if (n <= 5)
goto print;
}
return 0;
}
```

Any Question